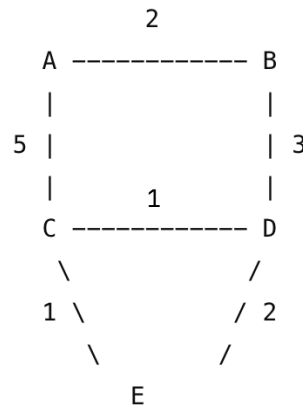


Exercise 2: Link State and Distance Vector Routing

Name: Dulthanee Pungkiatratsamee Student ID: 6710405320 Date: 16 Sep, 2026

Learning objectives: Apply Dijkstra and Bellman-Ford; construct forwarding tables; analyze convergence, failures, and count-to-infinity.

Network topology:



Part 1 — Link State Routing

Question 1 — Dijkstra's Algorithm

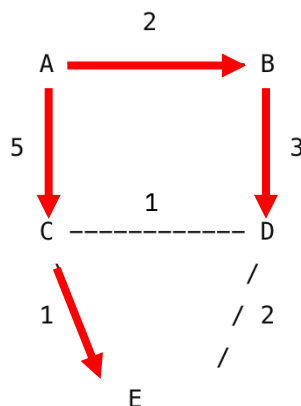
Starting at router A, execute Dijkstra's algorithm. Show every update and the node confirmed at each step.

Step	Confirmed Node	D(B), Prev.	D(C), Prev.	D(D), Prev.	D(E), Prev.
0	A	2, A	5, A	∞	∞
1	B		5, A	5, B	∞
2	C			5, B	6, C
3	D				6, C
4	E				

Question 2 — Shortest-Path Tree

Draw the shortest-path tree rooted at A, then list the shortest path and total cost.

SPT sketch:



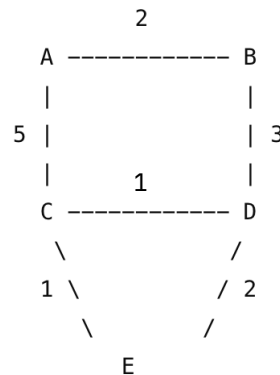
Destination	Shortest Path	Total Cost
B	A, B	2
C	A, C	5
D	A, B, D	5
E	A, C, E	6

Question 3 — Forwarding Table

Destination	Shortest Cost	Next Hop	Complete Path
B	2	B	A, B
C	5	C	A, C
D	5	B	A, B, D
E	6	C	A, C, E

Part 2 — Distance Vector Routing

Consider the same network. Initially, each router knows only the cost to its directly connected neighbors.



Question 4 — Bellman-Ford Update

Router A receives the following distance vectors from its neighbors:

From B:

A=2, B=0, C=4, D=3, E=5;

From C:

A=5, B=4, C=0, D=1, E=1.

Use the Bellman-Ford equation. Calculate router A's new distance to:

Show all calculations, including the neighbor through which the minimum-cost route is obtained.

Destination	Via B calculation	Via C calculation	Minimum Cost	Next Hop
B	$2+0 = 2$	$5+0 = 5$	2	B
C	$2+4 = 6$	$5+0 = 5$	5	C
D	$2+3 = 5$	$5+1 = 6$	5	B
E	$2+5 = 7$	$5+1 = 6$	6	C

Question 5 — Distance Vector Table

After receiving the distance vectors, construct router A's updated routing table.

Destination	Next Hop	Distance	Full Path
B	B	2	A, B
C	C	5	A, C
D	B	5	A, B, D
E	C	6	A, C, E

Part 3 — Routing Convergence

Question 6 — Link Failure

The C–D link fails. Explain separately what happens with Link State and Distance Vector routing.

Link State routing.

Your answer should describe:

- Which router(s) detect the failure.
- What information is flooded through the network.
- How routers update their link-state databases.
- How routers calculate new shortest paths.

Distance Vector routing.

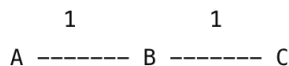
Your answer should describe:

- How routers learn about the failure.
- How updated distance vectors propagate.
- Why routers may temporarily have incorrect routing information.
- How routing eventually converges.

Link State	Distance Vector
Router C and D detect the failure.	Router C and D detect the failure first using keepalive mechanism. Then, other routers eventually learn about the failure from their neighbors.
The link states of router C and D, in which the link cost between them is updated to 'infinity'.	C and D update their own distance vectors and send them to their neighbors - C sends its updated vector to A and E. - D sends its updated vector to B and E. Neighbors then recompute their own vectors based on what they hear, and pass their updated vectors on to their neighbors, and so on.
Router A, B, and E replace the old entry for C-D link with a new one.	Because they can calculate the distance vector by mixing stale distances with new one, since each router doesn't know the whole network topology.
Each router independently reruns Dijkstra's algorithm with the new information.	Eventually, through repeated exchanges of updated vectors, convergence is reached once all routers' vectors stabilize.

Part 4 — Count-to-Infinity

Consider



Initially A reaches C through B at cost 2. The B–C link fails, but B receives stale information from A saying C is reachable at cost 2.

Question 7

1. Why can this create a routing loop?

Ans B would assume C is reachable via A, so it passes “packets going to C” to A. A would then pass them back to B since it was the only path. Thus, the loop is created.

2. Show how the estimated distance to C can increase over successive iterations.

[Ans](#)

Iteration	B's distance to C	A's distance to C
Before failure	1 (direct)	2 (via B)
After failure	∞ (direct link down)	2 (stale, via B)
1	B hears A=2 $\rightarrow$ B: $1+2 = 3$ (via A)	still 2
2	B advertises 3 $\rightarrow$ A hears it $\rightarrow$ A: $1+3 = 4$ (via B)	4
3	A advertises 4 $\rightarrow$ B: $1+4 = 5$ (via A)	4
4	B advertises 5 $\rightarrow$ A: $1+5 = 6$ (via B)	6
...	...	...

3. What is count-to-infinity?

[Ans](#) It's the failure scenario where, after a link/node goes down, routers repeatedly increment and exchange increasingly larger (but incorrect) distance estimates to the unreachable destination, instead of quickly recognizing it as unreachable.

4. Why does it occur in Distance Vector routing?

[Ans](#) DV routers only exchange distances, not the actual topology used to compute that distance. So, a router can't tell whether a neighbor's advertised route secretly depends on itself.

5. Name a mitigation approach.

[Ans](#) Split horizon with poison reverse: a router never advertises a route back to the neighbor it learned that route from, instead it advertises that destination as unreachable (cost ∞) to that neighbor.

Part 5 — Compare LS and DV

Question 8

Characteristic	Link State	Distance Vector
Basic algorithm	Each router builds a complete map of the network and independently computes shortest paths using Dijkstra's algorithm	Each router shares its own distance estimates with neighbors and computes paths using the Bellman-Ford algorithm
Information maintained	Full topology, all routers and links (with link costs)	Distance Vector: the cost to each known destination and which neighbor is the next hop
Information exchanged	Link-state packets (LSPs)	Entire distance vector as known by that router
Algorithm	Dijkstra's shortest-path algorithm	Bellman-Ford (distributed)
Topology knowledge	Global (every router knows the full network graph)	Local (a router only knows its neighbors' distances)
Routing updates	All-to-all routers	Sent only to <i>directly connected neighbors</i>
Convergence	Generally fast	Generally slower, since updates must propagate incrementally hop-by-hop
Routing loops	Rare	More common, since routers rely on neighbors' potentially stale or circular info
Count-to-infinity	Does not occur	Can occur
Example protocol	OSPF, IS-IS	RIP

Part 6 — Design Question

Question 9

You are designing a routing system for an enterprise with 500 routers requiring fast convergence, accurate topology knowledge, efficient routing, and hierarchical design. Choose LS or DV and justify your choice with at least three technical arguments. Your answer should discuss at least three advantages and/or disadvantages of your chosen approach.

[Ans](#) I choose LS with these reasons:

1. Fast convergence: Updates flood to all routers near-simultaneously; each computes shortest paths independently via Dijkstra. Meanwhile DV has slow hop-by-hop propagation.
2. Accurate topology knowledge: Every router holds a full, identical network map. DV only knows neighbor-reported distances with no path visibility.
3. Avoids loops/count-to-infinity: Independent computation from a consistent global view prevents the routing loops and count-to-infinity problems inherent to DV.
4. Native hierarchical support: OSPF's areas let routers keep detailed topology only within their area, with summarized routes exchanged via Area Border Routers, directly meeting the hierarchical design requirement and bounding database/computation size.

Challenge — Algorithmic Analysis

Question 10

For N routers and E links, compare LS and DV in information maintained, communication overhead, computation, convergence, and scalability. Explain why large-scale intra-domain routing generally favors Link State.

[Ans](#)

1. Comparison table

Characteristic	Link State	Distance Vector
Information maintained	$O(N + E)$ per router	$O(N)$ per router
Communication overhead	$O(N^2)$ messages network-wide	Sends full vector to neighbors only, but repeats each hop: can be $O(N)$ rounds of neighbor exchanges
Computation	Dijkstra's: $O(N \log N)$ per router	Bellman-Ford: $O(N \cdot E)$
Convergence	Fast	Slower, incremental hop-by-hop propagation
Scalability	Scales well with hierarchy (areas); per-router state grows but is bounded by area size	Degrades with N , slow convergence, loop/count-to-infinity risk grows with network diameter

2. Why large-scale intra-domain routing favors Link State

1. Convergence time grows with network diameter in DV
2. DV's count-to-infinity and loop risks scale with N .
3. LS supports hierarchical area design (e.g., OSPF)